Field Hours

CSCE 654 Supercomputing

Original notes for this site. Same topics as the catalog course. Not a reprint of another university's lectures, and not academic credit.

Lecture 1 · 90 minutes

Intensity and the ridge

You judge a kernel by arithmetic intensity: flops per byte moved. A node at 64 billion flops per second and 16 billion bytes per second has its ridge at 64/16, which is 4 flops per byte. Below 4, the kernel is bandwidth-bound even if every flop is a vector flop. Above 4, the flop peak is reachable in principle. You place the kernel on that roofline before you rewrite a line.

You count bytes the memory system actually moves, including a write of a dirty line, not only the operands written in the source. A 7-point stencil that updates one double of 8 bytes and does 7 flops has intensity 7/8 = 0.875, far under the ridge of 4. You will not make it compute-bound by unrolling. You cut bytes, by reusing a value already in cache, or you accept the bandwidth ceiling and stop tuning flops.

The ridge is peak flops divided by bandwidth. The kernel is a point, not a slogan.

This node

64 billion flops per second

16 billion bytes per second

Ridge at 4 flops per byte

A 7-point stencil

7 flops on one point

8 bytes moved

Intensity 0.875, under the ridge

Place the stencil

1. Peak is 64 billion flops per second and bandwidth is 16 billion bytes per second. The ridge is 64/16 = 4 flops per byte.

2. One updated double moves 8 bytes and the stencil does 7 flops. Intensity is 7/8 = 0.875.

3. 0.875 is below 4, so the ceiling is bandwidth. Raising the flop peak does not move this kernel until the bytes drop.

Problem 1. A kernel does 40 flops and moves 8 bytes. The ridge is 4. Which ceiling applies?

Answer. Intensity is 40/8 = 5 flops per byte, above 4. The flop peak is the ceiling, not the bandwidth line.

Problem 2. You double the peak to 128 billion flops per second and leave bandwidth at 16 billion bytes per second. What is the new ridge?

Answer. 128/16 = 8 flops per byte. A kernel that sat between 4 and 8 was compute-side of the old ridge and is bandwidth-bound on the new one.

Lecture 2 · 90 minutes

Vector lanes and the tail

You vectorize a loop by issuing one instruction on several lanes. A 256-bit register holds 4 doubles of 8 bytes, so one add updates 4 elements. The mechanism is a contiguous load into that register, aligned to 32 bytes when the instruction requires it. If iteration i depends on iteration i-1, the lanes are not independent and you do not issue the 4-wide add. You rewrite the dependence or you leave the loop scalar.

You check the remainder. A trip count of 10 is 2 vector iterations of 4 and a scalar tail of 2. Forgetting the tail drops elements 8 and 9. A branch inside the loop becomes a mask: lanes that fail the test still occupy the instruction, and you write back only the lanes you want. If the mask keeps only 1 lane of 4, you did the arithmetic of 4 and the useful work of 1.

Width 4 covers a multiple of 4. The tail is still your loop.

Count the trips -> Issue groups of 4 -> Run the scalar tail -> Mask partial lanes

Ten iterations, width 4

1. The vector width is 4 doubles. 10 = 2 times 4, plus a remainder of 2.

2. You issue 2 vector adds, covering elements 0 through 7.

3. The tail runs elements 8 and 9 as scalar code. Skip it and those two results stay at their old values.

Problem 1. The trip count is 16 and the width is 4. How many vector iterations, and how long is the tail?

Answer. 4 vector iterations and a tail of 0. 16 divided by 4 is 4 exactly, so nothing is left for scalar code.

Problem 2. A mask keeps 1 lane out of 4. How many lanes does the instruction occupy, and how many results do you keep?

Answer. The instruction occupies 4 lanes. You keep 1 result. The other 3 did arithmetic you discard.

Lecture 3 · 90 minutes

A team and a reduction

You mark a loop as shared-memory parallel work. A team of 8 threads splits iterations. The mechanism that keeps the answer right is a reduction on a sum: each thread accumulates a private partial, and the runtime adds those 8 partials at the join. If you instead let every thread write one shared total, the updates race and the total depends on the schedule. You do not call a parallel loop finished until you have named which variables are private.

You watch the schedule and the barrier. Static chunks are fine when each iteration costs the same. If the last chunk does 10 times the work, one of 8 threads holds that tail and the other 7 wait at the implicit barrier. A dynamic schedule hands the next chunk to a free thread and pays more dispatch. You remove the barrier with a nowait clause only when the next region does not read the values just written.

Private partials make the sum one answer. The barrier is where the other threads wait.

1. Fork a team of 8

2. Sum into private partials

3. Add the 8 partials

4. Barrier, unless nowait

Eight threads add 8000 numbers

1. The loop sums 8000 numbers. Eight threads each take 1000 of them into a private partial.

2. At the join the runtime adds those 8 partials. The total does not depend on which thread ran first.

3. A shared total updated inside the loop can lose additions. The private partial is the mechanism that prevents the race.

Problem 1. Eight threads add into one shared total with no reduction and no lock. Is the numeric result determined?

Answer. No. The threads race on that total. Eight private partials, added at the join, are the version with one result.

Problem 2. Seven of 8 threads finish, and the eighth holds a chunk that is 10 times heavier, with a barrier at the end of the loop. What are the other threads doing?

Answer. They wait at the implicit barrier. Team time is the heavy chunk, not the average chunk. A dynamic schedule can hand later chunks to the idle 7.

Lecture 4 · 90 minutes

Two ranks and a halo

You move bytes explicitly when the processes do not share memory. Rank 0 and rank 1 are two addresses. A send on rank 0 copies a buffer into the network; a matching receive on rank 1 copies it out. The match uses a tag and a communicator. If you send 8 doubles, that is 64 bytes, and the receive must ask for at least 64. A receive posted for 32 bytes does not describe this message.

You order the posts so you do not deadlock. If rank 0 and rank 1 each send before either receives, and the messages are larger than the eager buffer, both can wait forever. The safe pattern for a pair is: rank 0 sends then receives, rank 1 receives then sends. One exchange of a halo of 8 doubles is that pattern in both directions. You do not time the exchange until both sides have entered and left it.

A pair exchange deadlocks if both sides send first and the eager buffer cannot hold the message.

Rank 0

Send 8 doubles first

Then receive the halo

64 bytes on the wire

Rank 1

Receive 64 bytes first

Then send 8 doubles

Same tag, other way

Exchange two halos of 8 doubles

1. Rank 0 sends 8 doubles, 64 bytes, with tag 1, then posts a receive for rank 1's halo.

2. Rank 1 posts the receive for those 64 bytes first, then sends its own 8 doubles.

3. If both send first and the message exceeds the eager buffer, both sit in send. This order reaches both receives.

Problem 1. You send 8 doubles but post a receive of 32 bytes. What is wrong?

Answer. The message is 64 bytes. A 32-byte receive is smaller than the message. The receive count must be at least 64.

Problem 2. Rank 0 and rank 1 both send 8 doubles before either receives, and the eager buffer cannot hold the message. What happens?

Answer. Both block in send, and neither reaches its receive. That is a deadlock. One side must receive first.

Lecture 5 · 90 minutes

A stencil and its halo

You map the stencil onto the bytes you must move. A 5-point update reads the center and 4 neighbors and does 5 flops, writing one new double. On one rank, cached neighbors are cheap. Across ranks, the neighbor may be a ghost cell you do not own. You fill a halo of 1 layer by exchanging those ghosts before the sweep. Interior points then use the same 5-point formula as a serial code.

You count halo bytes against interior points. A strip of height 8 and width 1024 updates 8192 points. Two ghost edges move 2 times 1024 doubles, which is 16384 bytes. A strip of height 1 moves that same ghost traffic and updates only 1024 points, so you ship more than you compute. You thicken the owned rows until the ghost surface is a small fraction of the 8192-point interior.

Ghosts are not part of the answer. They exist so the 5-point formula can run unchanged.

Owned interior, height 8

One ghost layer on each edge

16384 bytes of ghosts

Then the 5-point update

Height 8, width 1024

1. The owned strip is height 8 and width 1024, so the interior count is 8 times 1024 = 8192 points.

2. Two ghost edges are 2 times 1024 doubles. At 8 bytes each, that is 16384 bytes moved before the sweep.

3. Height 1 would still move 16384 bytes and would update only 1024 points. You keep height 8 so 8192 updates share that traffic.

Problem 1. Height is 8 and width is 1024. How many interior points, and how many ghost bytes do two edges of doubles cost?

Answer. Interior points are 8 times 1024 = 8192. Ghost bytes are 2 times 1024 times 8 = 16384.

Problem 2. On one rank, a 5-point stencil still reads 4 neighbors. Why is that not the same bill as a halo?

Answer. The formula always reads 4 neighbors plus the center. On one rank those neighbors can hit cache. A halo is the copy of neighbors another rank owns, and you pay it on the network before the sweep.

Lecture 6 · 90 minutes

Barriers, flops, and scaling

You time a region with a barrier on both sides, otherwise rank 0's clock includes waiting you did not mean to include. You count flops from the algorithm, not from a hardware counter you have not defined. The 8192-point stencil at 5 flops per point is 40960 flops. If the timed region takes 0.001 seconds, you report 40960 / 0.001 = 40.96 million flops per second. You write the time and the flop count in the same note.

You name strong or weak scaling before you quote a speedup. Strong scaling keeps 8192 points fixed and adds ranks, so halos grow as a fraction of the work. Weak scaling gives each rank those same 8192 points and grows the grid. A drop from 8 seconds on 1 rank to 4 seconds on 2 ranks is speedup 2, strong. The same times on a weak study are a loss: work doubled, and time did not stay near 8.

The barriers define the region. The rate is flops you counted, divided by seconds you measured.

Barrier, then start -> Count algorithm flops -> Barrier, then stop -> Divide flops by seconds

40960 flops in a millisecond

1. There are 8192 points and 5 flops each, so the region does 40960 flops.

2. Barriers bracket the sweep. The clock reads 0.001 seconds.

3. The rate is 40960 / 0.001 = 40960000 flops per second, which is 40.96 million. A strong-scaling drop from 8 seconds to 4 seconds on 2 ranks is speedup 2.

Problem 1. Time falls from 8 seconds on 1 rank to 4 seconds on 2 ranks, with the same 8192 points. What is the strong-scaling speedup?

Answer. Speedup is 8/4 = 2. The point count stayed fixed, so the study is strong scaling, and the speedup is 2.

Problem 2. Each rank still owns 8192 points when you go from 1 rank to 2, and the time stays 8 seconds. Did weak-scaling efficiency hold?

Answer. Yes. Work per rank stayed the same and time stayed 8 seconds. If the time had risen above 8, efficiency would have dropped.

Lecture 7 · 90 minutes

Allreduce in a tree

You call a collective when every rank holds part of one answer. Allreduce adds one double from each rank and returns the total everywhere. On 8 ranks a binomial tree has depth 3: the count of partials goes 8, 4, 2, 1. That is 3 steps, not 7 sends. You use the library collective for this small payload. You drop to manual sends only after you time both on the same 8 ranks.

You separate latency from bandwidth in the call. An allreduce of 8 bytes is mostly latency, because 3 tree steps pay startup and the payload is tiny. An allreduce of 8 million doubles is mostly bandwidth, and the tree moves that buffer more than once. You reduce inside the rank first, then you allreduce a vector of length 4, instead of shipping every element.

Eight ranks, three steps. A short vector beats an allreduce of every element.

1. Reduce inside the rank

2. Allreduce length 4

3. Tree depth is 3

4. Use the total

Sum one double on 8 ranks

1. Each of 8 ranks holds one double. A binomial allreduce has depth 3, because the partials go 8, then 4, then 2, then 1.

2. A gather of the other 7 ranks straight to rank 0 is 7 sends in one direction, then more sends to broadcast the total.

3. If each rank already holds 4 local values, you sum them locally and allreduce a vector of length 4, not 4 separate calls.

Problem 1. How many binomial-tree steps does an allreduce use on 8 ranks, and how many sends does a one-direction gather to rank 0 use from the other ranks?

Answer. The tree has 3 steps. A gather to rank 0 uses 7 sends, one from each other rank. The tree is not those 7 sends.

Problem 2. Why is an allreduce of 8 bytes latency-bound, while an allreduce of 8 million doubles is not?

Answer. Three tree steps pay startup, and 8 bytes hide under that startup, so the call is latency. Millions of doubles make the bytes dominate, so the call is bandwidth. Reduce locally first when you can, and allreduce a vector of length 4.

Lecture 8 · 90 minutes

A kernel you can place

You can now place a scientific kernel on a machine and defend the placement. You compute intensity against a ridge, here 4 flops per byte on the node with peak 64 and bandwidth 16, and you say whether lanes of 4 doubles can matter. You choose a shared-memory team or explicit messages of 64-byte halos. You time with barriers, and you label the study strong or weak before you report a speedup.

You can also refuse a tuning step that cannot move the bound. Unrolling a stencil at intensity 0.875 does not cross a ridge of 4. Adding ranks on a strong problem of 8192 points helps only while the interior still dwarfs the halo of 16384 ghost bytes. You write the flop count, the byte count, and the seconds in one place, and you stop when the next change cannot touch the term that dominates.

Report these four numbers together. A speedup without them is an unlabeled clock.

Intensity against ridge 4

Vector width 4

Halo bytes versus interior

Seconds between barriers

Do not unroll past the ridge

1. The stencil does 7 flops and moves 8 bytes. Intensity is 0.875, and the ridge is 4, so unrolling cannot make the kernel compute-bound.

2. On a strong study the same 8192 points drop from 8 seconds on 1 rank to 4 seconds on 2 ranks. Speedup is 2.

3. You still owe the halo: 16384 ghost bytes. If that surface swallows the interior, the next rank will not repeat the speedup of 2.

Problem 1. The ridge is 4 flops per byte. A stencil does 5 flops and moves 8 bytes per point. A strong-scaling run goes from 8 seconds on 1 rank to 4 seconds on 2 ranks. What is the intensity, is the kernel compute-bound, and what is the speedup?

Answer. Intensity is 5/8 = 0.625 flops per byte, under 4, so the kernel is bandwidth-bound, not compute-bound. Speedup is 8/4 = 2. Extra vector flops do not cross the ridge; the time cut has to come from bandwidth or from less data per rank.

Problem 2. A strip is height 8 and width 1024, with two ghost edges of doubles. How many interior points share the 16384 ghost bytes?

Answer. 8192 interior points. That is 8 times 1024. The ghost bill stays 2 times 1024 times 8 = 16384 bytes. A height of 1 would put that same bill on only 1024 points.